

■ EventHub

Frontend Developer Roadmap — React JS

Complete guide: what to build, when to build it, and how long it takes

6

Phases
~6–9

Weeks
3

User Roles
30+

■ Before You Write a Single Line of Code

Read all requirements carefully	Understand the 3 actors, every feature, and the notes about WebSockets.
Draw the UI flow on paper	Sketch screens for: Landing → Login/Register → Home → Event Detail → Checkout → Dashboard (Organizer) → Admin Panel.
List all pages you need	Count ~12–15 distinct pages/views before touching code.
Plan your folder structure	Decide on src/pages/, src/components/, src/context/, src/hooks/, src/services/ upfront.
Set up Git from day 1	Create a repo, use branches (feature/phase-1, feature/auth, etc.), commit often.
Agree on API contract with backend team	Ask for the Swagger/OpenAPI spec or at least agree on endpoint names and response shapes before Phase 3.

■ The 6 Development Phases

Phase 1	Project Setup & Architecture ■ 2–3 days
	■ <i>Dependency: Nothing — do this first, before any feature work</i> ◆ Create React app with Vite: <code>npm create vite@latest eventhub -- --template react</code> ◆ Install core packages: <code>react-router-dom</code>, <code>axios</code>, <code>socket.io-client</code>, <code>react-query</code> (TanStack), <code>zustand</code> or <code>Context API</code> ◆ Install UI library: Tailwind CSS (recommended) or Material UI or <code>shadcn/ui</code> ◆ Set up folder structure: <code>/pages</code>, <code>/components</code>, <code>/layouts</code>, <code>/hooks</code>, <code>/context</code>, <code>/services</code>, <code>/utils</code> ◆ Create <code>.env</code> file with <code>VITE_API_URL=http://localhost:5000</code> and <code>VITE_SOCKET_URL=...</code> ◆ Configure <code>react-router-dom v6</code> with all route placeholders (even empty pages) ◆ Create a base <code>Axios</code> instance in <code>/services/api.js</code> with <code>baseUrl</code> and interceptors ◆ Set up a global <code>AuthContext</code> (<code>useContext</code>) to hold user state and token ◆ Configure <code>ESLint</code> + <code>Prettier</code> for code consistency ◆ Push skeleton project to Git ■ <i>Use Vite, not Create React App — it's much faster for development.</i> ■ <i>Define ALL routes upfront even if pages are empty. Prevents confusion later.</i>

Phase 2	Authentication & User Roles ■ 3–4 days
	■ <i>Dependency: Phase 1 complete. Can be done before backend is ready (use mock data)</i> ◆ Build Login page — form with email + password, POST to <code>/api/auth/login</code> ◆ Build Register page — two variants: Participant self-register, Organizer register (pending admin approval) ◆ Store JWT token in <code>localStorage</code> or <code>httpOnly</code> cookie (ask backend team) ◆ Create <code>PrivateRoute</code> component — redirects to login if not authenticated ◆ Create <code>RoleRoute</code> component — restricts pages by role (Admin/Organizer/Participant) ◆ Build Navbar with conditional rendering: show different links per role ◆ Handle token expiry: use <code>Axios</code> interceptors to auto-logout on 401 response ◆ Build a simple profile page (view/edit basic info) ◆ Test all 3 role flows: Admin login → Admin panel, Organizer login → Dashboard, Participant login → Home ■ <i>Mock the API responses with <code>json-server</code> or <code>MSW</code> (Mock Service Worker) while backend isn't ready.</i> ■ <i>Participants can VIEW events without login — make sure your <code>PrivateRoute</code> only blocks interaction (buy ticket, save, review), NOT viewing.</i>

Phase 3	Core Pages — Events (Read Side) ■ 4–5 days ■ <i>Dependency: Phase 2 complete. Need at least the GET /events and GET /events/:id endpoints from backend</i>
	◆ Build Landing / Home page — hero section, featured events grid, category filters ◆ Build All Events page — list/grid of event cards with search bar ◆ Implement search & filter: by location, category, date (controlled inputs → query params) ◆ Build Event Card component — reusable card showing image, title, date, venue, price, available tickets ◆ Build Event Detail page — full event info, organizer name, image, ticket count, buy button ◆ Handle 'Available Tickets' badge: show 'Sold Out' if availableTickets === 0 ◆ Build Category filter bar (Music, Tech, Sports, etc.) — clickable pills ◆ Add pagination or infinite scroll on the events list ◆ Make all pages fully responsive (mobile-first with Tailwind breakpoints) ◆ Handle loading states (skeleton cards) and error states (empty state UI) ■ <i>Build the Event Card as a reusable component from day 1 — you will use it everywhere.</i> ■ <i>Use React Query (useQuery) for all GET requests — handles caching, loading, and refetch automatically.</i>

Phase 4	Participant Features (Interactions) ■ 4–5 days ■ <i>Dependency: Phase 3 complete. Need ticket purchase, favorites, and reviews endpoints from backend</i>
	◆ Build ticket purchase flow: 'Buy Ticket' button → confirmation modal → POST /tickets ◆ On successful purchase: show unique digital ticket with QR code (use 'qrcode.react' package) ◆ Build 'My Tickets' page — list all tickets with QR codes for the logged-in participant ◆ Build Watchlist/Favorites: heart icon on event cards → POST/DELETE /favorites ◆ Build 'My Favorites' page — list of saved events ◆ Build Rate & Review form — star rating (1–5) + text review, only shown after event date has passed ◆ Display reviews on Event Detail page with average star rating ◆ Integrate WebSocket for real-time ticket updates: when availableTickets changes → update UI instantly ◆ Integrate WebSocket for notifications: show bell icon in navbar with unread count badge ◆ Build Notifications dropdown/panel — list of recent notifications from organizers ■ <i>For QR codes: npm install qrcode.react — just pass the ticket ID as the value.</i> ■ <i>Socket.io: connect once in App.jsx or AuthContext, then use in any component via a custom useSocket() hook.</i> ■ <i>Only show 'Buy Ticket' if user is logged in AND tickets are available. Otherwise show appropriate message.</i>

Phase 5	Organizer Dashboard & Admin Panel ■ 4–5 days
	■ <i>Dependency: Phase 4 mostly done. Need organizer CRUD and admin management endpoints</i> ◆ Build Organizer Dashboard layout with sidebar nav (My Events, Create Event, Analytics, Attachments) ◆ Build Create Event form — all fields: title, description, venue, category, date, price, total tickets, image upload ◆ Build Event list table (My Events) with Edit and Delete buttons ◆ Build Edit Event form — pre-filled with existing data ◆ Handle image upload: use FormData and multipart/form-data for event image ◆ Build Attachments page — upload/list event materials (PDF, schedules) ◆ Build Analytics view — show tickets sold, revenue (bar chart with recharts library) ◆ Implement real-time notification send: Organizer posts update → socket emits to all participants ◆ Build Admin Panel layout (separate dashboard for Admin role) ◆ Build 'Pending Accounts' page — list of Organizer registrations with Accept/Reject buttons ◆ Build 'Pending Events' page — list of submitted events with Accept/Reject and preview ◆ Accepted events become visible to participants (status toggle) ■ <i>Use recharts for analytics charts: npm install recharts — simple and React-friendly.</i> ■ <i>Protect all /organizer/* and /admin/* routes with RoleRoute from Phase 2.</i>

Phase 6	Polish, Testing & Deployment ■ 3–4 days
	■ <i>Dependency: All features complete. Full backend integration done</i> ◆ Go through every page and fix UI inconsistencies, spacing, and responsiveness ◆ Add proper error handling: API errors shown as toast notifications (react-hot-toast or sonner) ◆ Add form validation everywhere (react-hook-form + zod recommended) ◆ Add loading spinners / skeleton screens for all async operations ◆ Test all 3 user role flows end-to-end with real backend ◆ Test WebSocket features: real-time ticket count updates and notifications ◆ Test the complete ticket purchase → QR code generation → My Tickets flow ◆ Check for console errors, fix any broken routes or missing states ◆ Build for production: npm run build ◆ Deploy to Vercel or Netlify (free, easy, and connects to Git for auto-deploy) ◆ Write a short README.md with setup instructions for the college submission ■ <i>Vercel deployment: just connect your GitHub repo — zero configuration needed for Vite React apps.</i> ■ <i>Set environment variables (VITE_API_URL) in the Vercel dashboard under Project Settings.</i> ■ <i>Make sure CORS is configured correctly on the .NET backend before final testing.</i>

■ Timeline Overview

Phase	Focus	Duration	Backend Needed?
1	Setup & Architecture	2–3 days	No
2	Auth & Roles	3–4 days	Partially (can mock)
3	Events — Read Side	4–5 days	GET endpoints only
4	Participant Interactions	4–5 days	Yes — tickets, favorites, WS
5	Organizer & Admin Panels	4–5 days	Yes — full CRUD + admin
6	Polish & Deployment	3–4 days	Yes — full integration
	TOTAL	~6–9 weeks*	

* Working ~2–3 hours/day on weekdays. Full-time effort could cut this to 3–4 weeks.

■ Key npm Packages You'll Need

Package	Purpose	Phase
react-router-dom	Page routing and navigation	1
axios	HTTP requests to .NET backend API	
socket.io-client	Real-time WebSocket connection	1
@tanstack/react-query	Server state, caching, loading/error states	
react-hook-form + zod	Form handling and validation	2
zustand	Global state management (lightweight)	
tailwindcss	Utility-first CSS styling	1
react-hot-toast / sonner	Toast notifications for success/error	
qrcode.react	Generate QR codes for tickets	4
recharts	Analytics charts for Organizer dashboard	
react-dropzone	Drag-and-drop file upload for attachments	5
date-fns	Date formatting (event dates, filters)	

■ Recommended Folder Structure

```

src/ ■■■ assets/ # images, icons, fonts ■■■ components/ ■ ■■■ common/ # Button, Input,
Modal, Spinner, Badge... ■ ■■■ events/ # EventCard, EventGrid, EventFilters... ■ ■■■ tickets/
# TicketCard, QRCode display... ■ ■■■ reviews/ # StarRating, ReviewForm, ReviewList... ■ ■■■
layout/ # Navbar, Sidebar, Footer... ■■■ context/ ■ ■■■ AuthContext.jsx # user, token,
login(), logout() ■ ■■■ SocketContext.jsx # socket instance, useSocket() hook ■■■ hooks/ ■
■■■ useAuth.js # shortcut to AuthContext ■ ■■■ useEvents.js # React Query hooks for events ■
■■■ useNotifications.js ■■■ pages/ ■ ■■■ public/ # Landing, EventList, EventDetail, Login,
Register ■ ■■■ participant/ # MyTickets, Favorites, Profile ■ ■■■ organizer/ # Dashboard,
CreateEvent, EditEvent, Analytics ■ ■■■ admin/ # PendingAccounts, PendingEvents ■■■ services/
■ ■■■ api.js # Axios instance with interceptors ■ ■■■ authService.js # login, register,
logout API calls ■ ■■■ eventService.js # CRUD for events ■ ■■■ ticketService.js # purchase,
list tickets ■■■ utils/ ■ ■■■ formatDate.js ■ ■■■ constants.js # ROLES, CATEGORIES, STATUS
enums ■■■ App.jsx # All routes defined here ■■■ main.jsx

```

■ Real-Time WebSocket Guide (Required by Project)

What you need WebSockets for	1) Live ticket count update when someone buys a ticket – all viewers see 'Available: 47' change instantly. 2) Push notifications to participants when an organizer posts an update.
How to set it up (frontend)	Install: <code>npm install socket.io-client</code> Connect in <code>SocketContext.jsx</code> on login: <code>const socket = io(VITE_SOCKET_URL, { auth: { token } })</code> Expose via <code>useSocket()</code> hook.
Ticket count — listen in EventDetail.jsx	<code>socket.on('ticket:updated', ({ eventId, available }) => { if (eventId === currentEvent.id) setAvailableTickets(available) })</code>
Notifications — listen globally in Navbar	<code>socket.on('notification:new', (data) => { setNotifications(prev => [data, ...prev]) incrementBadgeCount() })</code>
Coordinate with backend team	Ask them what event names they emit (e.g., 'ticket:updated', 'notification:new') and what the payload looks like. Match exactly.

■ Working in Parallel with the Backend Team

■ Do BEFORE backend is ready	■ Do AFTER backend endpoints exist
• Full project setup and routing • All UI layouts and component designs • Login/Register forms (mock API) • Event list/detail pages with dummy JSON • All navigation and role-based routing • Form validation logic • QR code component (with dummy ID) • Dashboard layouts (Organizer & Admin) • Notification UI (bell icon, dropdown) • Socket context setup (connect later)	• Wire up real auth (JWT + refresh) • Connect event CRUD to real endpoints • Real ticket purchase with real ticket IDs • Real QR codes from ticket API response • Real favorites synced to database • Real reviews stored and fetched • Connect WebSocket events to live backend • Admin accept/reject calling real API • Organizer analytics from real data • File upload for event images/attachments

■ Final Tips for Success

■ Don't over-engineer early	Build simple first. A working basic version beats a perfect unfinished one.
■ Components > copy-paste	Every time you repeat UI, make it a component. You'll thank yourself in Phase 5.
■ Communicate with backend team daily	Mismatched API contracts are the #1 cause of frontend delays in team projects.
■ Test as you build	Don't leave all testing to Phase 6. Test each feature right after you build it.
■ Use React DevTools	Install the browser extension. It helps debug context, state, and re-renders instantly.
■ Version control everything	Commit every time something works. You can always go back to a working state.
■ Mobile responsiveness matters	Test on phone screen sizes from Phase 3 onwards — don't leave it to the end.